

PROJECT ARCHITECTURE

Technical Architecture – Stock Trading App (MERN Stack)

The Stock Trading App is developed using the MERN (MongoDB, Express.js, React.js, and Node.js) stack, which provides a scalable, secure, and responsive web application. The architecture follows a three-tier design consisting of the presentation layer, application layer, and database layer.

Frontend (Presentation Layer)

The frontend is built using React.js, along with HTML5, CSS3, and JavaScript. It provides a responsive and interactive user interface where users can register, log in, view live stock prices, buy or sell stocks, manage portfolios, and track transaction history. React communicates with the backend through REST APIs using Axios.

Backend (Application Layer)

The backend is developed using Node.js and Express.js. It handles business logic, user authentication, stock management, portfolio updates, transaction processing, and API requests. The backend validates user input, processes buy/sell operations, and sends appropriate responses to the frontend.

Database Layer

The application uses MongoDB, a NoSQL database, to store and manage application data. Collections include:

- Users
- Stocks
- Portfolio
- Transactions
- Watchlist

MongoDB ensures efficient storage and retrieval of data required for trading operations.

Authentication and Security

Security is implemented using:

- JWT (JSON Web Token) for secure user authentication.
- crypt for password hashing and encryption.
- Protected API routes to ensure only authorized users can access sensitive information.

External Services

The application can integrate with external APIs to:

- Fetch live stock market prices.

- Display real-time market updates.
- Send email or push notifications for important activities.

Request Flow

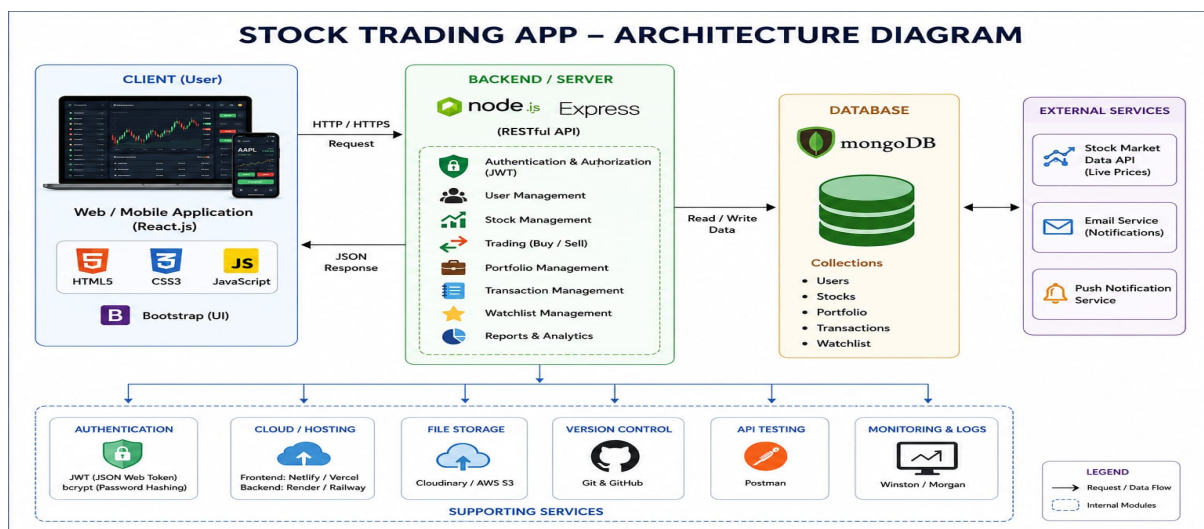
1. The user interacts with the React frontend.
2. The frontend sends HTTP requests to the Express.js server.
3. The backend processes the request and applies business logic.
4. Data is stored or retrieved from MongoDB.
5. The server returns a JSON response to the frontend.
6. The frontend displays the updated information to the user.

Technologies Used

Layer	Technology
Frontend	React.js, HTML5, CSS3, JavaScript, Bootstrap
Backend	Node.js, Express.js
Database	MongoDB
Authentication	JWT, bcrypt
API Communication	REST API, Axios
Version Control	Git & GitHub
API Testing	Postman

Summary

The Stock Trading App follows a client-server architecture using the MERN stack. React.js provides a dynamic user interface, Node.js and Express.js handle business logic and API services, MongoDB stores application data, and JWT with bcrypt ensures secure authentication. This architecture offers high performance, scalability, security, and maintainability, making it suitable for modern web-based stock trading applications.



ER Diagram Explanation – Stock Trading App

The Entity Relationship (ER) Diagram represents the database structure of the Stock Trading App. It shows the main entities, their attributes, and the relationships between them. The database is designed to manage users, stocks, portfolios, transactions, and watchlists efficiently.

1. User Entity

Attributes

- **userId (PK):** Unique identifier for each user.
- **name:** User's full name.
- **email:** Registered email address.
- **password:** Encrypted user password.
- **balance:** Available account balance for trading.

2. Stock Entity

Attributes

- **stockId (PK):** Unique stock identifier.
- **companyName:** Name of the company.
- **symbol:** Stock market symbol (e.g., TCS, INFY).
- **currentPrice:** Current market price of the stock.

3. Portfolio Entity

Attributes

- **portfolioId (PK):** Portfolio record identifier.
- **userId (FK):** References the User table.
- **stockId (FK):** References the Stock table.
- **quantity:** Number of shares owned.

4. Transaction Entity

Attributes

- **transactionId (PK):** Unique transaction identifier.
- **userId (FK):** References the User table.
- **stockId (FK):** References the Stock table.
- **type:** Buy or Sell.
- **quantity:** Number of shares traded.
- **amount:** Total transaction amount.

5. Watchlist Entity

Attributes

- watchlistId (PK): Unique watchlist identifier.
- userId (FK): References the User table.
- stockId (FK): References the Stock table

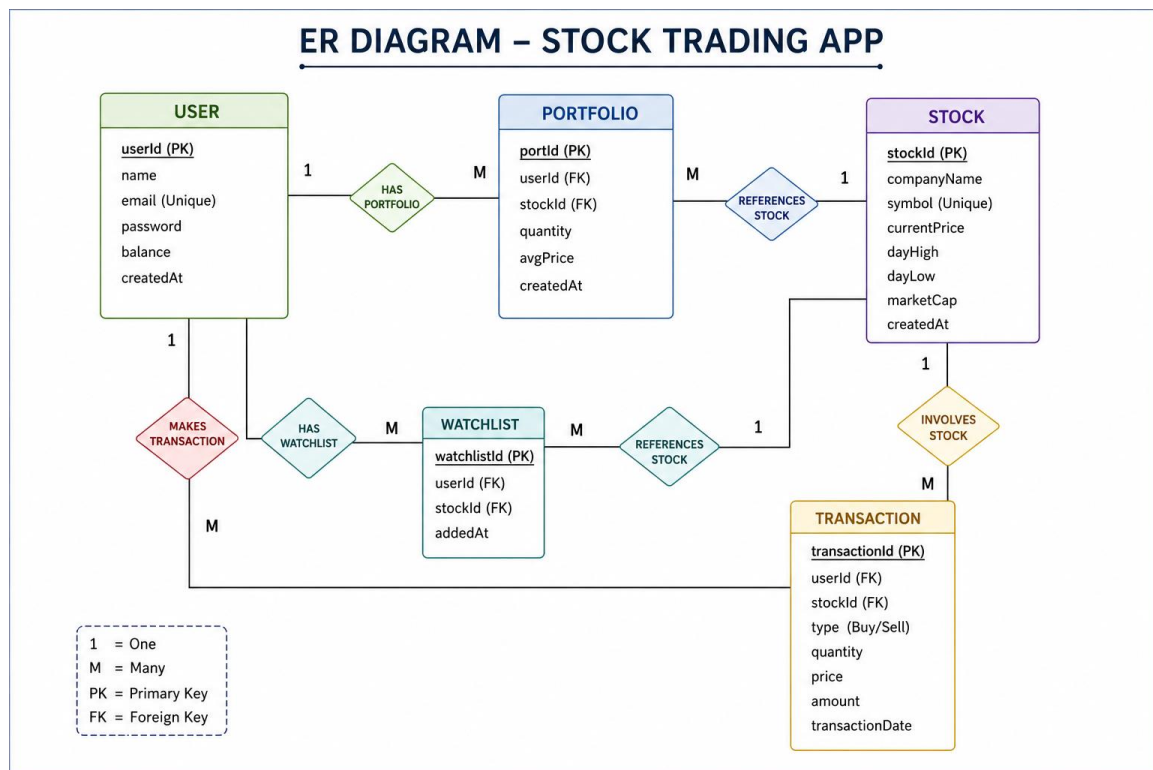
Relationships

- User → Portfolio (1:Many): One user can have multiple portfolio records, but each portfolio record belongs to only one user.
- Stock → Portfolio (1:Many): One stock can appear in the portfolios of many users, while each portfolio record refers to one stock.
- User → Transaction (1:Many): A user can perform multiple buy and sell transactions, but each transaction belongs to one user.
- Stock → Transaction (1:Many): One stock can have many transactions, while each transaction is associated with one stock.
- User → Watchlist (1:Many): A user can add multiple stocks to their watchlist.
- Stock → Watchlist (1:Many): A stock can appear in the watchlists of many users.

Primary Key (PK) and Foreign Key (FK)

Primary Key (PK): A unique identifier for each record in a table.

Foreign Key (FK): A field that links one table to another, maintaining relationships between entities.



Stock Trading App – Features, Roles & Responsibilities

Features

User Features:

- User Registration and Secure Login
- Interactive Dashboard with Market Overview
- View and Search Live Stock Prices
- Buy and Sell Stocks
- Portfolio Management
- Transaction History
- Watchlist Management
- Profile Management and Secure Logout

Admin Features:

- Manage Users
- Add, Update, and Delete Stock Information
- Monitor Trading Activities and Transactions
- Update Stock Prices and Market Data
- Generate Reports and Maintain System Data

Roles & Responsibilities

User:

- Register and log in securely.
- Search stocks and view market prices.
- Buy and sell stocks.
- Maintain portfolio and watchlist.
- View transaction history and update profile.

Admin:

- Manage users and stock records.
- Verify and monitor transactions.
- Update stock information.
- Generate reports and maintain application security.
- Ensure smooth operation of the trading platform.

Summary: The Stock Trading App provides secure stock trading, portfolio management, and real-time market access. Users perform trading operations, while administrators manage the platform, maintain data accuracy, and ensure reliable system performance.

Stock Trading App – User Flow

Overview

The user flow describes how a user interacts with the Stock Trading App from registration to managing investments. It ensures a simple, secure, and efficient trading experience.

User Flow Steps

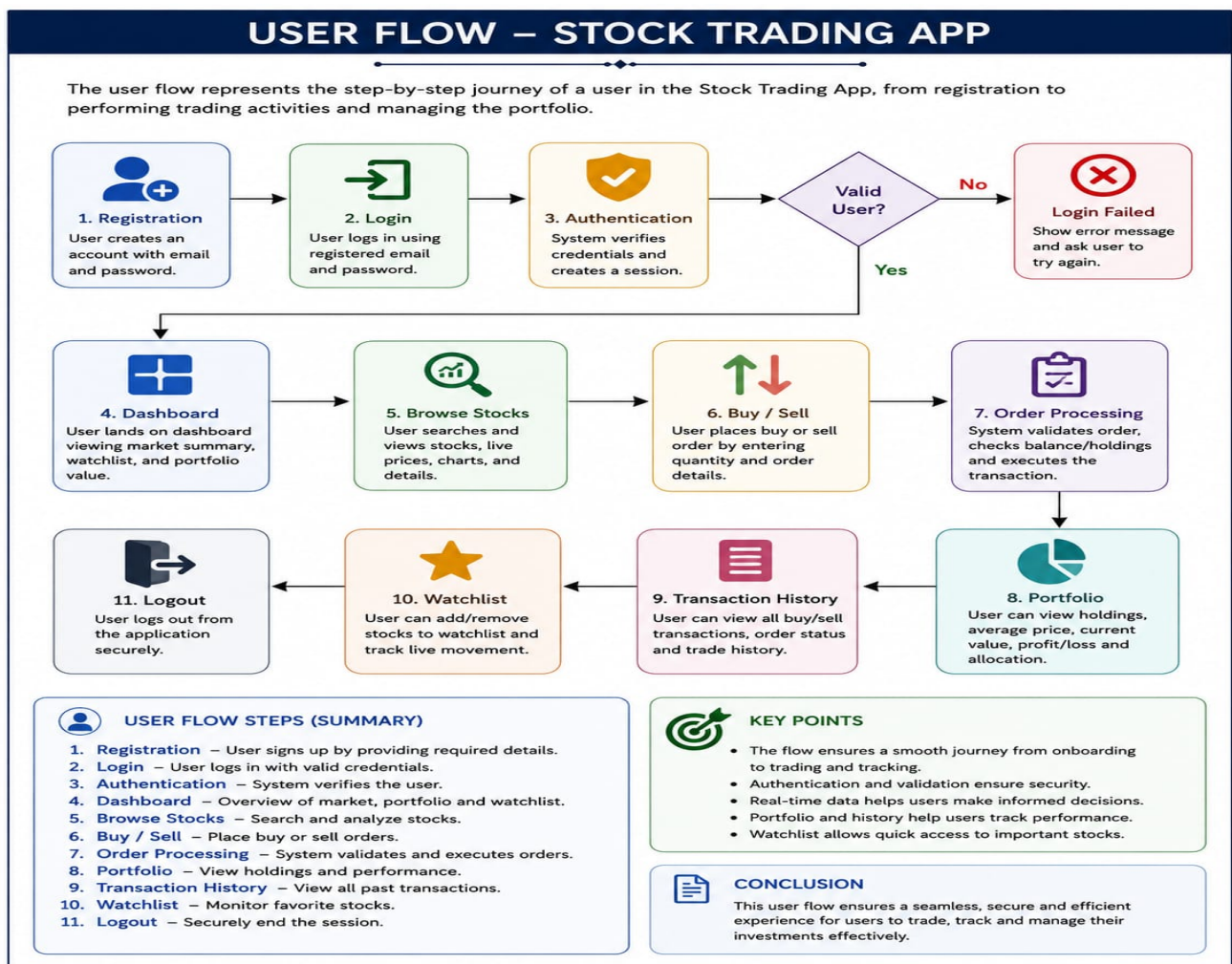
1. User registers a new account or logs in securely.
2. The dashboard displays market trends and available stocks.
3. The user searches and views stock details.
4. The user buys or sells stocks based on market prices.
5. The portfolio is automatically updated after each transaction.
6. All buy/sell activities are stored in the transaction history.
7. The user adds favorite stocks to the watchlist for monitoring.
8. The user updates profile information if required.
9. The user logs out securely, ending the session.

Flow Diagram

Start → Register/Login → Dashboard → View/Search Stocks → Buy/Sell Stocks → Portfolio Updated → Transaction History → Watchlist/Profile → Logout → End

Conclusion

This user flow provides a seamless process for secure authentication, stock trading, portfolio management, and account maintenance, ensuring an intuitive user experience.



MVC Pattern – Stock Trading App

Overview

The Stock Trading App uses the MVC (Model-View-Controller) architecture to separate the application into Model, View, and Controller components. This improves scalability, maintainability, and code organization.

Model

Manages application data, communicates with MongoDB, and handles CRUD operations for Users, Stocks, Portfolio, Transactions, and Watchlist.

View

Built with React.js, HTML, CSS, and JavaScript. It displays the interface, accepts user input, and sends requests to the Controller.

Controller

Processes requests, validates data, implements business logic, communicates with the Model, and returns responses to the View.

Workflow

User → View → Controller → Model → MongoDB → Controller → View → User

Advantages

- Separation of concerns
- Easy maintenance
- Better scalability
- Reusable code
- Faster development

